

Expense Tracker

Design Document

1. The problem

- Every bank sends an SMS the moment money leaves or enters your account. (mainly for India)
- That information is already sitting on your phone, but it is scattered across hundreds of messages, so nobody actually uses it.
- Tracking spending manually means typing in every purchase, which almost nobody keeps up for more than a week.
- Apps that do this automatically exist, but they upload your financial data to their own servers, and you have no way of knowing what happens to it after that.
- The gap: an app that reads those messages and organises them for you, without your financial history ending up on somebody else's computer.

2. What the app does

- Runs quietly in the background on an Android phone.
- Notices when a bank message arrives and pulls out the useful bits: how much, where, and what kind of spending it was.
- Sorts each transaction into a category — food, travel, rent, and so on.
- Keeps everything stored on the phone itself, not in the cloud.
- Gets more accurate the more you use it, because it remembers your corrections.
- Can show your data on a computer screen when you want a bigger view, without permanently storing it anywhere.

3. How it works

Spotting the right messages

- Most SMS on a phone are junk: one-time passwords, offers, delivery updates.
- A simple word check runs first — messages containing words like "debited", "credited" or "UPI" are worth a closer look, everything else is ignored immediately.
- This is deliberately cheap and fast, so the phone is not doing heavy work all day.

Understanding the message

- Messages that pass the word check get sent to an AI model, which reads the text and pulls out the details in a structured form.
- This replaces the traditional approach of writing a separate rule for every bank's message format, which breaks every time a bank changes its wording.
- The AI is also told to reject anything that is not a real transaction, so promotional messages mentioning "cashback credited" do not become fake entries.

Avoiding double-counting

- Banks often send more than one message for a single purchase — one when the payment is authorised, another when it actually settles.

- Most bank messages contain a reference number. If two messages share the same reference number, they are the same transaction, and only the later one is kept.
- When there is no reference number, the app compares the account, the amount and the timing.
- If it cannot be certain, it does not guess. The transaction is flagged and shown to the user to confirm.

Sorting into categories

- The app ships with common categories: income, food, travel, utilities, groceries, entertainment, investments, housing.
- Users can add their own — someone with a car might want fuel and servicing tracked separately from general travel.
- Custom income types work the same way: rent received, dividends, money lent or repaid.
- Deleting a category does not delete its transactions. They are moved to another category instead.

Learning your habits

- If the same shop shows up repeatedly, the app asks once whether it should always be filed the same way, then remembers the answer.
- After that, those transactions are sorted instantly without needing the AI at all — so the app gets faster and cheaper the longer it is used.
- Any correction the user makes is fed back in as context, so future guesses reflect how that particular person thinks about their spending.
- Anything the app genuinely cannot place goes into a review list rather than being quietly dropped.

Viewing it on a computer

- A phone screen is a poor place to look at months of spending.
- The phone displays a QR code containing a temporary connection detail and a one-time key.
- The computer scans it, the two devices connect directly to each other, and the data is sent across encrypted.
- The web page holds the data in memory only. Close the tab and it is gone — nothing is saved on the computer or on any server.

4. The privacy approach

- Transaction history lives only on the phone.
- The user supplies their own AI provider key, so the AI usage is under their own account and their own terms, not the app developer's.
- The server involved is a pass-through. It reads the message, returns the result, and stores nothing.
- There are no user accounts, no logins, and no user database anywhere in the system.
- The connection between phone and computer is temporary and expires quickly, so a stray screenshot of the QR code is not a lasting risk.

5. Technology used

Phone app

- Kotlin — the standard language for building Android apps.
- Jetpack Compose — the toolkit used to build the screens.
- Room / SQLite — the database that lives on the phone and holds all transactions.

- WorkManager — handles work that needs to happen in the background, including retrying when the phone has no signal.
- Notification Listener — the Android feature that lets the app see incoming messages.

Server side

- Node.js with TypeScript — runs the two small services.
- A pass-through service that forwards message text to the AI and returns the result, storing nothing.
- A connection helper that introduces the phone and the computer to each other, then steps out of the way.

Computer dashboard

- React — builds the web page.
- WebRTC — the technology that lets two devices talk directly to each other, the same one used by video calling.
- QR scanning through the computer's camera to pick up the connection details and key.

6. How data moves

Everyday use

- A bank message arrives on the phone.
- The word check decides whether it is worth processing. Most messages stop here.
- Anything that passes is saved to a waiting list on the phone, so nothing is lost if there is no internet.
- When there is a connection, the message text is sent out, read by the AI, and the structured result comes straight back.
- The phone checks it against known shops and known duplicates.
- The finished transaction is saved on the phone and appears in the app.

Sending to a computer

- The user chooses to export, and the phone generates a QR code.
- The computer scans it and the two devices establish a direct connection.
- The data is encrypted on the phone, sent across, and unlocked on the computer using the key from the QR code.
- The dashboard is displayed. Closing the tab discards everything.

7. Ideas for later

- GPS location capturing when an SMS is sent to help with categorizing the transaction, for example, daily 50 rs near a train station at 9 am to a random UPI ID can be safely categorized as a travel expense. More context and meta data (this was initially the core concept)
- Automatically summarising the user's corrections into a short profile, so the app builds its own understanding of the user's preferences instead of relying on rules alone.
- Wider merchant knowledge, so common shops are recognised without needing the AI at all.
- Richer spending reports and trends on the computer dashboard.

8. Known issues and open questions

Still to be worked out

- The list of trigger words has not been finalized. It needs to be built from real bank messages rather than guessed, because banks phrase things differently from each other.
- The same applies to duplicate detection timings and to how shop names should be tidied up before comparing them.
- The final list of default categories should reflect real spending patterns rather than a generic template.

Technical limitations

- iPhones do not allow apps to read messages at all, so this is Android only. This is a restriction from Apple, not a design choice.
- Android phone manufacturers aggressively shut down background apps to save battery. This can cause the app to miss messages, and the fix differs from brand to brand.
- Setting up message access requires the user to dig into phone settings manually. It cannot be done with a simple pop-up.
- Scheduled background work on Android is best-effort. It can be delayed by the phone's own power saving.

Accepted trade-offs

- Message text does briefly leave the phone to be read by the AI. Nothing is stored, but this is not the same as fully offline. (local LLM can be used, but massive battery drain + slower and less accurate)
- Every message processed costs a small amount against the user's AI account. This reduces over time as the app learns recurring shops.
- Because everything is stored on the phone only, losing the phone means losing the data. An export option is needed to cover this. (can use a simple SQL db and give user a choice to use local or cloud storage)
- Cash spending is invisible to the app, since no bank message is generated. (add manual input, but most people do not do that)

Risks to watch

- If a bank changes its message wording, transactions may silently stop being recorded, with no error shown.
- Promotional messages can look like real transactions and slip through.
- The AI can occasionally misread a message, so a manual review path is essential rather than optional.